HABIB UNIVERSITY

Data Structures & Algorithms

CS/CE 102/171 Spring 2025

Instructor: Maria Samad

Date: February 3rd, 2025

Quiz # 1

Time Allowed: 45 minutes

Max Points: 25

Name: _____

QUESTION 1: (20 marks)

You are given an input string called ***str1*** containing 'n' characters including blank spaces, where 'n' is at least 2, and another string called ***word*** containing no spaces with length < n. You are also given a stack called ***StackChars*** of size 'n', which is initially empty. The program removes the specified word from the stack, while keeping all the other words in the same order of their appearance in string. Assume 'str1' and 'word' are correctly given to you with the allowed length, so you may use them directly in your algorithm, without having to do any error checking or taking input. However, do declare them in the algorithm to define what they are, including the stacks and/or other ADTs that you may use for your program.

DO NOT TRY TO ACHIEVE THE TASK IN ANY WAY OTHER THAN THE FOLLOWING STEPS:

1. It reads each character of string, ***str1*** and adds them to the stack, ***StackChars*** one at a time, such that the last letter of the string appears at the top of the stack.
2. Once all letters are in the stack, you CANNOT use ***str1*** again anywhere in the program. If it is being used for any reason – simple or complex, the remaining algorithm will NOT be graded
3. Using the stack of letters, i.e. ***StackChars***, you should update the same stack such that after removal of instances of ***word***, the letters in the stack appear in the same order as they were input in Step 1.

For example,

- Assume ***str1*** = "**I am not sad not bot"**, and given word to be removed = **"not"**, so as a result of first task, ***StackChars*** = ['I', ' ', 'a', 'm', ' ', 'n', 'o', 't', ' ', 's', 'a', 'd', ' ', 'n', 'o', 't', ' ', 'b', 'o', 't']
- Once ***StackChars*** is filled, remove the appearance of all **"not"** from the stack with stack resulting in: ***StackChars*** = ['I', ' ', 'a', 'm', ' ', ' ', 's', 'a', 'd', ' ', ' ', 'b', 'o', 't']

You must implement the above tasks by writing an algorithm for each part.

Restrictions:

- Do not **HARD CODE** your design for the given example. The algorithm should work for any valid sized string.
- Assume the length of string is 'n', where n > 1, so there will always be at least two letters in any given string, resulting in Stack of size at least 2
- Assume None represents empty slots in all the data structures used for this question
- You may use additional variables, but as minimum as possible
- You are **NOT** allowed to use **any** additional **ADT**, including ListADT
- **DO NOT USE ANY OTHER DATA STRUCTURES INCLUDING ListADT/Python lists apart from the ones permitted to use**
- **REMEMBER: Stacks and Queues are non-iterable and non-traversing data structures, so you CANNOT and MUST NOT do indexing within the data structures to access the elements**

QUESTION 2: (5 marks)

We have the following 2D-array stored in memory: ***arr1*** = [[1, 2, 3, 4, 5], [6, 19, 20, 21, 22], [11, 12, 7, 9, 11]]. Assume each number is 16 bits in size, and one location of memory is 8-bits wide. Calculate the address of the element, arr1[1][2]

SHOW ALL THE NECESSARY STEPS/FORMULA TO ANSWER THE ABOVE QUESTION, OTHERWISE, NO MARKS WILL BE AWARDED EVEN FOR CORRECT ANSWERS!